

Kapitel 26. Typinformation

Innehållsförteckning

[Varför dynamisk typinformation](#)[Hur används typkonvertering](#)[Typinformation](#)[Missbruk av typinformation](#)

Detta kapitel behandlar ett ganska nytt tillägg till standarden för C++, nämligen möjligheten att kunna få reda på ett objekts typ dynamiskt.

Varför dynamisk typinformation

Dynamisk typinformation kan vara praktisk att ha då man vill veta exakt vilken typ av objekt man hanterar just då. Om vi har en klasshierarki med klasserna `A`, `B` och `C` som är definierade enligt:

```
class A { ... };  
class B : public A { ... };  
class C : public B { ... };
```

Klassen `B` har några metoder som inte `A` har och `C` har några metoder som inte `B` har. Vi kan även anta att vi har en t.ex. en metod som tar som parameter en pekare till ett objekt av typen `A`. Objekten som egentligen skickas till metoden kan vara av alla tre typer. Vi utför någon viss virtuell metod som är definierad för alla objekt som skickas, men vi skulle även vilja exekvera någon extra metod för objekten av typen `B` och `C`. Det enda som vi hittills har kunnat göra är att t.ex. definiera en metod i `A` som returnerar objektets typ, t.ex. en sträng. Det är dock väldigt anti-OO, och därför finns det funktioner för att utföra dynamisk konvertering av objekt till en annan klass (om det är möjligt).

Typkonvertering fungerar *endast* med klasser som har *virtuella funktioner*. Dessa typer av klasser är även de enda typerna av klasser där man skall referera till ett objekt via en pekare till en basklass.